Extensions in-depth

What is an Extension?

An extension is, in its simplest form, just a folder with a config file (`extension.toml`). The Extension system will find that extension and if it's enabled it will do whatever the config file tells it to do, which may include loading python modules, **Carbonite** plugins, shared libraries, applying settings etc.

There are many variations, you can have an extension whose sole purpose is just to enable 10 other extensions for example, or just apply some settings.

Extension in a single folder

Everything that an extension has should be contained or nested within its root folder. This is a convention we are trying to adhere to. Looking at other package managers, like those in the Linux ecosystem, the content of packages can be spread across the filesystem, which makes some things easier (like loading shared libraries), but also creates many other problems.

Following this convention makes the installation step very simple - we just have to unpack a single folder.

A typical extension might look like this:

```
[extensions-folder]
└── omni.appwindow
    ├─bin
    │  └──windows-x86_64
    │      └──debug
    │          └── omni.appwindow.plugin.dll
    ├──config
    │   └── extension.toml
    └──omni
        └──appwindow
            ├── _appwindow.cp37-win_amd64.pyd
            └── __init__.py
```

This example contains a **Carbonite** plugin and a python module (which contains the bindings to this plugin).

Extension Id

Extension id consists of 3 parts: `[ext_name]-[ext_tag]-[ext_version]`:

- `[ext_name]`: Extension name. Extension folder or kit file name.
- `[ext_tag]`: Extension tag. Optional. Used to have different implementations of the same extension. Also part of folder or kit file name.
- `[ext_version]`: Extension version. Defined in `extension.toml`. Can also be part of folder name, but ignored there.

Extension id example: `omni.kit.commands-1.2.3-beta.1`. Extension name is `omni.kit.commands`. Version is `1.2.3-beta.1`. Tag is empty.

Extension Version

Version is defined in `[package.version]` config field.

Semantic Versioning is used. A good example of valid and invalid versions: link

In short, it is `[major].[minor].[patch]-[prerelease]`. Express compatibility with version change:

- For breaking change increment major version
- For backwards compatible change increment minor version
- For bugfixes increment patch version

Use `[prerelease]` part to test a new version e.g. `1.2.3-beta.1`

Extension Package Id

Extension package id is extension id plus build metadata. `[ext_id]+[build_meta]`.

One extension id can have 1 or multiple packages to support different targets. It is common for binary extensions to have packages like:

- `[ext_id]+wx64.r` - windows-x86_64, release config.
- `[ext_id]+lx64.r` - linux-x86_64, release config.
- `[ext_id]+lx64.d` - linux-x86_64, debug config.
- ...

Python version or kit version can also denote different targets. Refer to `[package.target]` section of extension config.

Single File Extensions

Single file Extensions are supported - i.e. Extensions consisting only of a config file, without any code. In this case, the name of the config will be used as the extension ID. This is used to make a top-level extension which we call an **app**. They are used as an application entry point, to unroll the whole extension tree.

The file extension can be anything (`.toml` for instance), but the recommendation is to name them with the `.kit` file extension, so that it can be associated with the `kit.exe` executable and launched with a single click.

```
[extensions-folder]
└── omni.exp.hello.kit
```

App Extensions

When `.kit` files are passed to `kit.exe` they are treated specially:

This: `> kit.exe C:/abc/omni.exp.hello.kit`

Is the same as: `> kit.exe --ext-path C:/abc/omni.exp.hello.kit --enable omni.exp.hello`

It adds this `.kit` file as an extension and enables it, ignoring any default app configs, and effectively starting an app.

Single file (`.kit` file) extensions are considered apps, and a “launch” button is added to it in the UI of the extension browser. For regular extensions, specify the keyword: `package.app = true` in the config file to mark your extension as an app that users can launch.

App extensions can be published, versioned, etc. just like normal extensions. So for instance if the `omni.exp.hello` example from above is published, we can just run Kit as:

`> kit.exe omni.exp.hello.kit`

Kit will pull it from the registry and start.

Extension Search Paths

Extensions are automatically searched for in specified folders.

Core **Kit** config `kit-core.json` specifies default search folders in `/app/exts/foldersCore` setting. This way **Kit** can find core extensions, it also looks for extensions in system-specific documents folders for user convenience.

To add more folders to search paths there are a few ways:

1. Pass `--ext-folder [PATH]` CLI argument to kit.
2. Add to array in settings: `/app/exts/folders`
3. Use the `omni::ext::ExtensionManager::addPath` API to add more folders (also available in python).

To specify a direct path to a specific extension use the `/app/exts/paths` setting or the `--ext-path [PATH]` CLI argument.

Folders added last are searched first. This way they will be prioritized over others, allowing the user to override existing extensions.

Example of adding an extension search path in a kit file:

```
[settings.app.exts]
folders.'++' = [
    "C:/hello/my_extensions"
]
```

Extensions Source Linking: Dev Paths

Often, when developing an app or an extension you want to link a local version of one of it's dependencies. All the methods above can be used to add another extension search paths, but there is a preferred way to do this: **Dev Paths**.

Instead of `--/app/exts/paths` and `--/app/exts/folders`, use `--/app/exts/devPaths` and `--/app/exts/devFolders`. For those paths extension system will ignore all version checks and prioritize them over other paths. This is useful for development, when you want to link a local version of an extension, or when you want to test a new version of an extension before publishing it.

A typical example would be to use a user file, `user.toml`. Create `deps/user.toml` file in this repo with the search to path to your repo added to `app/exts/devFolders` setting, e.g.:

```
[app.exts.devFolders]
"++" = [ "[path_to_project]/_build/windows-x86_64/release/exts",]
```

A `repo source` tool can also be used to create and edit `user.toml`. Provide a path to a repo or a direct path to an extension(s):

> `repo source link [repo_path]` - If repo produces **Kit** extensions add them to `deps/user.toml` file.

> `repo source link [ext_path]` - If the path is a **Kit** extension or folder with **Kit** extensions add to `deps/user.toml` file.

You can always find out where an extension is coming from in *Extension Manager* by selecting an extension and hovering over the open button.

You can also find it in the log, by looking either for `registered` message for each extension or `About to startup:` when it starts.

Custom Search Paths Protocols

Both folders and direct paths can be extended to support other url schemes. If no scheme is specified, they are assumed to be local filesystem. The extension system provides APIs to implement custom protocols. This way an extension can be written to enable searching for extensions in different locations, for example: git repos.

E.g. `--ext-folder foo://abc/def` – The extension manager will redirect this search path to the implementor of the `foo` scheme, if it was registered.

Extension Discovery

The Extension system monitors any specified extension search folders (or direct paths) for changes. It automatically syncs all changed/added/removed extensions. Any subfolder which contains an `extension.toml` in the root or `config` folder is considered to be an extension.

The subfolder name uniquely identifies the extension and is used to extract the extension name and tag: `[ext_name]-[ext_tag]`

```
[extensions-folder]
└── omni.kit.example-gpu-2.0.1-stable.3+cp37
            ├── extension.toml
            └── ...
```

In this example we have an extension `omni.kit.example-gpu-2.0.1-stable.3+cp37`, where:

- **name:** `omni.kit.example`
- **tag:** `gpu` (optional, default is "")

The version and other information (like supported platforms) are queried in the extension config file (see below). They may also be included in the folder name, which is what the system does with packages downloaded from a remote registry, but that is not "the ground truth" for that data. So in this example anything could have been after the "gpu" tag, e.g. `omni.kit.example-gpu-whatever`.

Extension Dependencies

When a Kit-based application starts, it discovers all extensions and does nothing with them until some of them are enabled, whether via config file or API. Each extension can depend on other extensions and this is where the whole application tree can unroll. The user may enable a high-level extension like **omni.usd_viewer** which will bring in dozens of others.

An extension can express dependencies on other extensions using **name**, **version** and optionally **tag**. It is important to keep extensions versioned properly and express breaking changes using Semantic Versioning.

This is a good place to grasp what **tag** is for. If extension `foo` depends on `bar`, you might implement other versions of `bar`, like: `bar-light`, `bar-prototype`. If they still fulfill the same API contract and expected behavior, you can safely substitute `bar` without `foo` noticing. In other words, if the extension is an interface, **tag** is the implementation.

The effect is that just enabling some high-level extensions like omni.kit.window.script_editor will expand the whole dependency tree in the correct order without the user having to specify all of them or worry about initialization order.

One can also substitute extensions in a tree with a different version or tag, towards the same end-user experience, but having swapped in place a different low-level building block.

When an extension is enabled, the manager tries to satisfy all of its dependencies by recursively solving the dependency graph. This is a difficult problem - If dependency resolution succeeds the whole dependency tree is enabled in-order so that all dependents are enabled first. The opposite is true for disabling extensions. All extensions that depend on the target extension are disabled first. More details on the dependency system can be found in the C++ unit tests: `source/tests/test.unit/ext/TestExtensions.cpp`

A Dependency graph defines the order in which extensions are loaded - it is sorted topologically. There are, however, many ways to sort the same graph (think of independent branches). To give finer control over the startup order, the `order` parameters can be used. Each extension can use `core.order` config parameter to define its order and the order of dependencies can also be overridden with `order`` param in [[dependencies]]` section. Those with lower order will start sooner. If there are multiple extensions that depend on one extension and are trying to override this order then the one that is loaded last will be used (according to the dependency tree). In summary - the dependency order is always satisfied (or extensions won't be started at all if the graph contains cycles) and soft ordering is applied on top using config params.

Specifying Dependency Version

Semantic Versioning is used to specify a version. It establishes a common convention for what is compatible between different versions of a package.

Versions are considered compatible if their left-most non-zero major/minor/patch component is the same. For example, `1.0.3` and `1.1.0` are considered compatible, and thus it should be safe to update from the older release to the newer one. However, an update from `1.1.0` to `2.0.0` would not be allowed to be made automatically. This convention also applies to versions with leading zeros. For example, `0.1.0` and `0.1.2` are compatible, but `0.1.0` and `0.2.0` are not. Similarly, `0.0.1` and `0.0.2` are not compatible.

SemVer requirement is used by default, or can be explicitly marked by prefixing version with `^` symbol, e.g. both `version = "1.2.3"` and `version = "^1.2.3"` are SemVer-compatible requirements.

Other version requirements can be specified using the following operators:

Requirement	Example	Equivalence	Description
Caret	`1.2.3` or `^1.2.3`	`>=1.2.3, <2.0.0`	Any SemVer-compatible version of at least the given value.

Requirement	Example	Equivalence	Description
Tilde	`~1.2`	`>=1.2.0, <1.3.0`	Minimum version, with restricted compatibility range.
Equals	`=1.2.3`	`=1.2.3`	Exactly the specified version only.
Comparison	`>1.1`	`>=1.2.0`	Naive numeric comparison of specified digits.

When multiple extensions specify a dependency for a common extension, the resolver attempts to find a version that satisfies all of the requirements. If no such version exists, the resolver will fail to enable the extension.

Resolver also attempts to use the latest compatible version of a dependency but prefers local (previously chosen and downloaded) over a new remote. This is to avoid an application upgrading many of the extensions unexpectedly, just because the registry was synced.

See Application Dependencies Management to learn how to manage the dependencies of an application.

Pre-release versions

Pre-release versions (e.g. `1.2.3-beta.1`) are supported. The main use case is to test new features without disrupting anything. Thus they have the lowest priority in version ordering. You can publish a pre-release version of an extension, but no one will pick it up unless they explicitly specify it as a dependency.

Here is an example of how versions are ordered from highest to lowest priority when resolving dependencies, all stable versions go first, then pre-release versions:

```
2.0.0
1.2.3
1.2.0
1.0.0
3.0.0-beta.2
3.0.0-beta.1
1.2.0-alpha.feature.test.3
```

This way when application extensions are updated, it will prefer stable versions over pre-release ones.

A rule of thumb is to specify a pre-release version exactly (e.g. `=1.2.3-beta.1` or `--enable 1.2.3-beta.1`) to test it. From a version requirements perspective, pre-release versions go just before the stable version, for example:

- `^1.0.0` matches `1.0.1-beta.1` but not `1.0.0-beta.1`
- `^1.4.2-beta.5` matches `1.4.2-beta.6` but not `1.4.2-beta.4`
- `<1.0.0` matches `1.0.0-beta` but not `1.0.0`

Optional Dependencies

An extension can specify that a dependency is optional. This is done by adding `optional = true` to the dependency section in the config file.

When extension depends on another extension optionally, it claims that it can work without it. Typically by checking if the extension is enabled or not at some point and behaving differently. This is useful for extensions that are not strictly required, but provide additional functionality if available.

By default, Kit will not enable optional dependncies. E.g. say `foo` depends on `bar` optionally. If we run `kit.exe --enable foo`, it will not enable `bar`. If we run `kit.exe --enable foo --enable bar`, it will enable both and will guarantee that `bar` is enabled before `foo`.

```
[[dependencies]]
"bar" = { optional=true }
```

It can get much more complex, many extensions can depend on `bar` optionally through various dependency chains and `bar` can bring many more extnesions. Extension solver identifies if the whole dependency tree is optional and won't enable any of them. But it is enough to have it specified non-optional in one of the branches to enable the whole tree. In that case all requirements (like version) must be satisfied even if they are optional or Kit will fail to resolve and enable extensions.

Once optional dependency is enabled it is treated the same as any other dependency. If we disable it at runtime (through API call) or in the UI, all extensions that depend on it will be disabled as well.

Extension configuration file (extension.toml)

An Extension config file can specify:

1. Dependencies to import
2. Settings
3. A variety of metadata/information which are used by the Extension Registry browser (for example)

TOML is the format used. See this short toml tutorial. Note in particular the `[[]]` TOML syntax for arrays of objects and quotes around keys that contain special symbols(e.g. `"omni.physx"`).

The config file should be placed in the root of the extension folder or in a `config` subfolder.

> **Note**
>
> All relative paths in configs are relative to the extension folder. All paths accept tokens. (like `${platform}`, `${config}`, `${kit}` etc). More info: Tokens.

There are no mandatory fields in config, so even with an empty config, the extension will be considered valid and can be enabled - without any effect.

Next, we will list all the config fields the extension system uses, though a config file may contain more. The Extension system provides a mechanism to query config files and hook into itself. That allows us to extend the extension system itself and add new config sections. For instance `omni.kit.pipapi` allows extensions to specify pip packages to be installed before enabling them. More info on that: Hooks. That also means that **typos or unknown config settings will be left as-is and no warning will be issued**.

Config Fields

`[core]` section

For generic extension properties. Used directly by the Extension Manager core system.

`[core.reloadable]` (default: `true`)

Is the extension **reloadable**? The Extension system will monitor the extension's files for changes and try to reload the extension when any of them change. If the extension is marked as **non-reloadable**, all other extensions that depend on it are also non-reloadable.

`[core.order]` (default: `0`)

When extensions are independent of each other they are enabled in an undefined order. An extension can be ordered to be before (negative) or after (positive) other extensions

`[package]` section

Contains information used for publishing extensions and displaying user-facing details about the package.

`[package.version]` (default: `"0.0.0"`)

Extension version. This setting is required in order to publish extensions to the remote registry. The Semantic Versioning concept is baked into the extension system, so make sure to follow the basic rules:

- Before you reach `1.0.0`, anything goes, but if you make breaking changes, increment the minor version.
- After `1.0.0`, only make breaking changes when you increment the major version.
- Incrementing the minor version implies a backward-compatible change. Let's say extension `bar` depends on `foo-1.2.0`. That means that `foo-1.3.0`, `foo-1.4.5`, etc.. are also suitable and can be enabled by extension system.
- Use version numbers with three numeric parts such as `1.0.0` rather than `1.0`.
- Prerelease labels can also be used like so: `1.3.4-beta`, `1.3.4-rc1.test.1`, or `1.3.4-stable`.

`[package.title]` default: `""`

User-facing package name, used for UI.

`[package.description]` default: `""`

User facing package description, used for UI.

`[package.category]` (default: `""`)

Extension category, used for UI. One of:

- `animation`

- `graph`
- `rendering`
- `audio`
- `simulation`
- `example`
- `internal`
- `other`

`[package.app]` (default: `false`)

Whether the extension is an App. Used to mark extension as an app in the UI. Adds a "Launch" button to run kit with only this extension (and its dependents) enabled. For single-file extensions (`.kit` files), it defaults to `true`.

`[package.feature]` (default: `false`)

Extension is a Feature. Used to show user-facing extensions, suitable to be enabled by the user from the UI. By default, the app can choose to show only those feature extensions.

`[package.toggleable]` (default: `true`)

Indicates whether an extension can be toggled (i.e enabled/disabled) by the user from the UI. There is another related setting: `[core.reloadable]`, which can prevent the user from disabling an extension in the UI.

`[package.authors]` (default: `[""]`)

Lists people or organizations that are considered the "authors" of the package. Optionally include email addresses within angled brackets after each author.

`[package.repository]` (default: `""`)

URL of the extension source repository, used for display in the UI.

`[package.keywords]` (default: `[""]`)

Array of strings that describe this extension. Helpful when searching for it in an Extension registry.

`[package.changelog]` (default: `""`)

Location of a CHANGELOG.MD file in the target (final) folder of the Extension, relative to the root. The UI will load and show it. We can also insert the content of that file inline instead of specifying a filepath. It is important to keep the changelog updated when new versions of an extension are released and published.

For more info on writing changelogs refer to Keep a Changelog

`[package.readme]` (default: `""`)

Location of README file in the target (final) folder of an extension, relative to the root. The UI will load and show it. We can also insert the content of that file inline instead of specifying a filepath.

`[package.preview_image]` (default: `""`)

Location of a preview image in the target (final) folder of extension, relative to the root. The preview image is shown in the “Overview” of the extension in the Extensions window. A screenshot of your extension might make a good preview image.

`[package.icon]` (default: `""`)

Location of the icon in the target (final) folder of extension, relative to the root. Icon is shown in Extensions window. Recommended icon size is 256x256 pixels.

`[package.target]`

This section is used to describe the target platform this extension runs on - this is fairly arbitrary, but can include:

- Operating system
- CPU architecture
- Python version
- Build configuration

- Kit version

The Extension system will filter out extensions that doesn't match the current environment/platform. This is particularly important for extensions published and downloaded from a remote Extension registry.

Normally you don't need to fill this section in manually. When extensions are published, this will be automatically filled in with defaults, more in Publishing Extensions. But it can be overriden by setting:

`package.target.config` (default: `["*"]`) – Build config, e.g. `"debug"`, `"release"`.

`package.target.platform` (default: `["*"]`) – Build platform, e.g. `"windows-x86_64"`, `"linux-aarch64"`.

`package.target.python` (default: `["*"]`) – Python version, e.g. `"cp310"` (cpython 3.10). Refer to **PEP 0425**.

`package.target.kit` (default: `["*"]`) – Kit version (major.minor), e.g. `"101.0"`, `"102.3"`.

`package.target.kitHash` (default: `["*"]`) – Kit git hash (8 symbols), e.g. `"abcdabcd"`

Wildcards can be used. A full example:

```
[package.target]
config = ["debug"]
platform = ["linux-*", "windows"]
python = ["*"]
```

Kit target is special, it compares the current **Kit** version is greater or equal to the one written in `target.kit`.

For example, if you run with **Kit** `105.1.1`, those will be suitable:

```
target.kit = ["105.0.0"]
target.kit = ["105.1.1"]
target.kit = ["104.0"]
```

Those are incompatible:

```
target.kit = ["105.1.2"]
target.kit = ["105.2"]
target.kit = ["106.0.0"]
```

`[package.writeTarget]`

This section can be used to explicitly control if `[package.target]` should be written. By default it is written based on rules described in Extension Publishing. But if for some `[target]` a field is set, such as `package.writeTarget.[target] = true/false`, that tells explicitly whether it should automatically be filled in.

For example if you want to target a specific kit version (the one used for publishing an extension) to make the extension work with that version or higher, set:

```
[package]
writeTarget.kit = true
```

Or if you want your extension to work for all python versions, write:

```
[package]
writeTarget.python = false
```

The list of known targets is the same as in the `[package.target]` section: `config`, `platform`, `python`, `kit`, `kitHash`.

`[dependencies]` section

This section is used to describe which extensions this extension depends on. The extension system will guarantee they are enabled before it loads your extension (assuming that it doesn't fail to enable any component). One can optionally specify a version and tag per dependency, as well as make a dependency optional.

Each entry is a **name** of another extension. It may or may not additionally specify: **tag**, **version**, **optional**, **exact**:

```
"omni.physx" = { version="1.0", "tag"="gpu" }
"omni.foo" = { version="3.2" }
"omni.cool" = { optional = true }
```

Note that it is highly recommended to use versions, as it will help maintain stability for extensions (and the whole application) - i.e if a breaking change happens in a dependency and dependents have not yet been updated, an older version can still be used instead. (only the *major* and *minor* parts are needed for this according to semver).

`version` (default: `""`, any version) – version of the dependency. See Specifying Dependency Version.

`optional` (default: `false`) – will mean that if the extension system can't resolve this dependency the extension will still be enabled. So it is expected that the extension can handle the absence of this dependency. Optional dependencies are not enabled unless they are a non-optional dependency of some other extension which is enabled, or if they are enabled explicitly (using API, settings, CLI etc).

`exact` (default: `false`) – only an exact version match of extension will be used. It is equal to using `=` in front of the version, e.g. `version="=1.2.3"`. Prefer to use `=` instead of `exact=true`, it is kept for backwards compatibility.

`order` (default: `None`) – override the `core.order` parameter of an extension that it depends on. Only applied if set.

`[python]` section

If an extension contains python modules or scripts this is where to specify them.

`[[python.module]]`

Specifies python module(s) that are part of this extension. Multiple can be specified. Take notice the `[[]]` syntax. When an extension is enabled, modules are imported in order. Here we specify 2 python modules to be imported (`import omni.hello` and `import omni.calculator`). When modules are scheduled for import this way, they will be reloaded if the module is already present.

Example:

```
[[python.module]]
name = "omni.hello"

[[python.module]]
name = "omni.calculator"
path = "."
public = true
```

`name` (required) – python module name, can be empty. Think of it as what will be imported by other extensions that depend on you:

```
import omni.calculator
```

`public` (default: `true`) – If public, a module will be available to be imported by other extensions (extension folder is added to sys.path). Non-public modules have limited support and their use is not recommended.

`path` (default: `"."`) – Path to the root folder where the python module is located. If written as relative, it is relative to extension root. Think of it as what gets added to `sys.path`. By default the extension root folder is added if any `[[python.module]]` directive is specified.

`searchExt` (default: `true`) – If true, imports said module and launches the extensions search routine within the module. If false, only the module is imported.

By default the extension system uses a custom fast importer. Fast importer only looks for python modules in extension root subfolders that correspond to the module namespace. In the example above it would only look in `[ext root]/omni/**`. If you have other subfolders that contain python modules you at least need to specify top level namespace. E.g. if you have also `foo.bar` in `[ext root]/foo/bar.py`:

```
[[python.module]]
name = "foo"
```

Would make it discoverable by fast importer. You can also just specify empty name to make importer search all subfolders:

```
[[python.module]]
path = "."
```

Example of that is in `omni.kit.pip_archive` which brings a lot of different modules, which would be tedious to list.

`[[python.scriptFolder]]`

Script folders can be added to `IAppScripting`, and they will be searched for when a script file path is specified to executed (with –exec or via API).

Example:

```
[[python.scriptFolder]]
path = "scripts"
```

`path` (required) – Path to the script folder to be added. If the path is relative it is relative to the extension root.

`[native]` section

Used to specify **Carbonite** plugins to be loaded.

`[[native.plugin]]`

When an Extension is enabled, the Extension system will search for **Carbonite** plugins using `path` pattern and load all of them. It will also try to acquire the `omni::ext::IExt` interface if any of the plugins implements it. That provides an optional entry point in C++ code where your extension can be loaded.

When an extension is disabled it releases any acquired interfaces which may lead to plugins being unloaded.

Example:

```
[[native.plugin]]
path = "bin/${platform}/${config}/*.plugin"
recursive = false
```

`path` (required) – Path to search for **Carbonite** plugins, may contain wildcards and Tokens).

`recursive` (default: `false`) – Search recursively in folders.

`[[native.library]]` section

Used to specify shared libraries to load when an Extension is enabled.

When an Extension is enabled the Extension system will search for native shared libraries using `path` and load them. This mechanism is useful to “preload” libraries needed later, avoid OS specific calls in your code, and the use of `PATH`/`LD_LIBRARY_PATH` etc to locate and load DSOs/DLLs. With this approach, we just load the libraries needed directly.

When an extension is disabled it tries to unload those shared libraries.

Example:

```
[[native.library]]
path = "bin/${platform}/${config}/foo.dll"
```

`path` (required) – Path to search for shared libraries, may contain wildcards and Tokens.

`[settings]` section

Everything under this section is applied to the root of the global **Carbonite** settings (`carb.settings.plugin`). In case of conflict, the original setting is kept.

It is good practice to namespace your settings with your extension name and put them all under the `exts` root key, e.g.:

```
[settings]
exts."omni.kit.renderer.core".compatibilityMode = true
```

Note

Quotes are used here to distinguish between the `.` of a toml file and the `.` in the name of extension.

An important detail is that settings are applied in reverse order of extension startup (before any extensions start) and they don't override each other. Therefore a parent extension can specify settings for child extensions to use.

`[[env]]` section

This section is used to specify one or more environment variables to set when an extension is enabled. Just like settings, env vars are applied in reverse order of startup. They don't by default override if already set, but override behavior does allow parent extensions to override env vars of extensions they depend on.

Example:

```
[[env]]
name = "HELLO"
value = "123"
isPath = false
append = false
override = false
platform = "windows-x86_64"
```

`name` (required) – Environment variable name.

`value` (required) – Environment variable value to set.

`isPath` (default: `false`) – Treat value as path. If relative it is relative to the extension root folder. Tokens can also be used as within any path.

`append` (default: `false`) – Append value to already set env var if any. Platform-specific separators will be used.

`override` (default: `false`) – Override value of already set env var if any.

`platform` (default: `""`) – Set only if platform matches pattern. Wildcards can be used.

`[fswatcher]` section

Used to specify file system watcher used by the Extension system to monitor for changes in extensions and auto reload.

`[fswatcher.patterns]`

Specify files that are monitored.

`include` (default: `["*.toml", "*.py"]`) – File patterns to include.

`exclude` (default: `[]`) – File patterns to exclude.

Example:

```
[fswatcher.patterns]
include = ["*.toml", "*.py", "*.txt"]
exclude = ["*.cache"]
```

`[fswatcher.paths]`

Specify folders that are monitored. FS watcher will use OS specific API to listen for changes on those folders. You can use that setting that limit amount of subscriptions if your extensions has too many folders inside. Simple path as string pattern matching is used, where '*' means any amount of symbols. Patterns like `*/123*` will match `abc/123/def`, `abc/123`, `abc/1234`. '/' path separator is used on all platforms. Matching paths are relative to extension root, they begin and end with '/'.

`include` (default: `["*/config/*", "*/./*"]` and python modules) – Folder path patterns to include.

`exclude` (default: `["*/__pycache__/*", "*/.git/*"]`) – Folder path patterns to exclude.

Example:

```
[fswatcher.paths]
include = ["*/config"]
exclude = ["*/data*"]
```

`[[test]]` section

This section is read only by the testing system (`omni.kit.test` extension) when running per-extension tests. Extension tests are run as a separate process where only the tested extension is enabled and runs all the tests that it has. Usually this section can be left empty, but extensions can specify additional extensions (which are not a part of their regular dependencies, or when the dependency is optional), additional cmd arguments, or filter out (or in) any additional stdout messages.

Each `[[test]]` entry will run a separate extension test process.

Extension tests run in the context of an app. An app can be empty, which makes extension test isolated and only its dependencies are enabled. Testing in an empty app is the minimal recommended test coverage. Extension developers can then opt-in to be tested in other apps and fine-tune their test settings per app.

Example:

```
[[test]]
name = "default"
enabled = true
apps = [""]
args = ["-v"]
dependencies = ["omni.kit.capture.viewport"]
pythonTests.include = ["omni.foo.*"]
pythonTests.exclude = []
cppTests.libraries = ["bin/${lib_prefix}omni.appwindow.tests${lib_ext}"]
timeout = 180
parallelizable = true
unreliable = false
profiling = false
pyCoverageEnabled = false
waiver = ""
stdoutFailPatterns.include = ["*[error]*", "*[fatal]*"]
stdoutFailPatterns.exclude = [
    "*Leaking graphics objects*",  # Exclude graphics leaks until fixed
    "*[carb.crashreporter-breakpad.plugin] [previous crash]*", # Ignore crashes
]
setup_fn = "omni/foo/bar/tests/setup.py:ext_test_setup"
```

`name` – Test process name. If there are multiple `[[test]]` entries, this name must be unique.

`enabled` – If tests are enabled. By default it is true. Useful to disable tests per platform.

`apps` – List of apps to use this test configuration for. Used in case there are multiple `[[test]]` entries. Wildcards are supported. Defaults to `[""]` which is an empty app.

`args` – Additional cmd arguments to pass into the extension test process.

`dependencies` – List of additional extensions to enable when running the extension tests.

`pythonTests.include` – List of tests to run. If empty, python modules are used instead (`[[python.module]]`), since all tests names start with module they are defined in. Can contain wildcards.

`pythonTests.exclude` – List of tests to exclude from running. Can contain wildcards.

`cppTests.libraries` – List of shared libraries with C++ doctests to load.

`timeout` (default: `180`) – Test process timeout (in seconds).

`parallelizable` (default: `true`) – Whether the test processes can run in parallel relative to other test processes.

`unreliable` (default: `false`) – If marked as unreliable, test failures won't fail a whole test run.

`profiling` (default: `true`) – Collects and outputs Chrome trace data via carb profiling for CPU events for the test process.

`pyCoverageEnabled` (default: `false`) – Collects python code coverage using Coverage.py.

`waiver` – String explaining why an extension contains no tests.

`stdoutFailPatterns.include` – List of additional patterns to search stdout for and mark as a failure. Can contain wildcards.

`stdoutFailPatterns.exclude` – List of additional patterns to search stdout for and exclude as a test failure. Can contain wildcards.

`setup_fn` – Path to a python function to run before the test process. Format is `<module_path>:<function_name>`, where `<module_path>` is a relative path to the extension root. The function will be passed the test object.

`[documentation]` section

This section is read by the `omni.kit.documenation.builder` extension, and is used to specify a list of markdown files for in-app API documentation and offline sphinx generation.

Example:

```
[documentation]
pages = ["docs/Overview.md"]
menu = "Help/API/omni.kit.documentation.builder"
title = "Omni UI Documentation Builder"
```

`pages` – List of .md file paths, relative to the extension root.

`menu` – Menu item path to add to the popup in-app documentation window.

`title` – Title of the documentation window.

`[deprecation]` section

For deprecating extensions.

`[deprecation.warning]` (default: `""`)

A deprecation warning message to show when the extension is enabled. Message will be prefixed with “DEPRECATION WARNING:” and logged to warning log. If non-empty extension is considered deprecated.

Config Filters

Any part of a config can be filtered based on the current platform, build configuration or a setting value. Use `"filter:[filter]".[value]` syntax:

- For platform, use `"filter:platform"."[platform_name]"`, e.g. `"filter:platform"."windows-x86_64"`.
- For os, use `"filter:os"."[os_name]"`, e.g. `"filter:os"."windows"`.
- For architecture, use `"filter:arch"."[architecture]"`, e.g. `"filter:arch"."x86_64"`.
- For build configuration, use `"filter:config"."[build_config]"`, e.g. `"filter:config"."debug"`.
- For settings, use `"filter:setting".[setting_path]."value:[value]"`, where `[setting_path]` is dot separated path to a setting (normal TOML path) and `[value]` is a setting value to compare to. E.g. `"filter:setting".app.mode "value:demo"`.

filter	values
platform	`windows-x86_64`, `linux-x86_64`, `linux-aarch64`
os	`windows`, `linux`
arch	`x86_64`, `aarch64`
config	`debug`, `release`
setting	`path.to.setting."value:[value]`

Anything under those keys will be merged on top of the tree they are located in (or filtered out if it doesn’t apply).

To understand, here are some examples:

```
[dependencies]
"omni.foo" = {}
"filter:platform"."windows-x86_64"."omni.fox" = {}
"filter:platform"."linux-x86_64"."omni.owl" = {}
"filter:config"."debug"."omni.cat" = {}
"filter:setting".app.wolf."value:true"."omni.wolf" = {}
"filter:setting".app.wolf."value:false"."omni.bear" = {}
```

After loading that extension on a Windows debug build with `--/app/wolf=true` setting set, it would resolve to:

```
[dependencies]
"omni.foo" = {}
"omni.fox" = {}
"omni.cat" = {}
"omni.wolf" = {}
```

Note

You can debug this behavior by running in debug mode, with `--/app/extensions/debugMode=1` setting and looking into the log file.

Example

Here is a full example of an `extension.toml` file:

```toml
[core]
reloadable = true
order = 0

[package]
version = "0.1.0"
category = "Example"
feature = false
app = false
title = "The Best Package"
description = "long and boring text.."
authors = ["John Smith <jsmith@email.com>"]
repository = "https://github.com/NVIDIA-Omniverse/kit-app-template"
keywords = ["banana", "apple"]
changelog = "docs/CHANGELOG.md"
readme = "docs/README.md"
preview_image = "data/preview.png"
icon = "data/icon.png"

# writeTarget.kit = true
# writeTarget.kitHash = true
# writeTarget.platform = true
# writeTarget.config = true
# writeTarget.python = true

[dependencies]
"omni.physx" = { version="1.0", "tag"="gpu" }
"omni.foo" = {}

# Modules are loaded in order. Here we specify 2 python modules to be imported (
[[python.module]]
name = "hello"
path = "."
public = false

[[python.module]]
name = "omni.physx"

[[python.scriptFolder]]
path = "scripts"

# Native section, used if extension contains any Carbonite plugins to be loaded
[[native.plugin]]
path = "bin/${platform}/${config}/*.plugin"
recursive = false  # false is default, hence it is optional

# Library section. Shared libraries will be loaded when the extension is enabled
[[native.library]]
path = "bin/${platform}/${config}/foo.dll"

# Settings. They are applied on the root of global settings. In case of conflict
[settings]
exts."omni.kit.renderer.core".compatibilityMode = true

# Environment variables. Example of adding "data" folder in extension root to PA
[[env]]
name = "PATH"
value = "data"
```

```
isPath = true
append = true
platform = "windows-x86_64"

# Fs Watcher patterns and folders. Specify which files are monitored for changes
[fswatcher]
patterns.include = ["*.toml", "*.py"]
patterns.exclude = []
paths.include = ["*"]
paths.exclude = ["*/__pycache__*", "*/.git*"]

[documentation]
pages = ["docs/Overview.md"]
menu = "Help/API/omni.kit.documentation.builder"
title = "Omni UI Documentation Builder"
```

Extension Enabling/Disabling

Extensions can be enabled and disabled at runtime using the provided API. The default **Base Editor** application comes with an Extension Manager UI which shows all the available extensions and allows a user to toggle them. An App configuration file can also be used to control which extensions are to be enabled.

You may also use command-line arguments to the Kit executable (or any Omniverse App based on Kit) to enable specific extensions:

Example: `> kit.exe --enable omni.kit.window.console --enable omni.kit.window.extensions`

`--enable` adds the chosen extension to the “enabled list”. The command above will start only extensions needed to show those 2 windows.

Python Modules

Enabling an extension loads the python modules specified and searches for children of :class:`omni.ext.IExt` class. They are instantiated and the `on_startup` method is called, e.g.:

`hello.py`

```
import omni.ext

class MyExt(omni.ext.IExt):
    def on_startup(self, ext_id):
        pass

    def on_shutdown(self):
        pass
```

When an extension is disabled, `on_shutdown` is called and all references to the extension object are released.

Native Plugins

Enabling an extension loads all Carbonite plugins specified by search masks in the `native.plugin` section. If one or more plugins implement the `omni::ext::IExt` interface, it is acquired and the `onStartup` method is called. When an extension is disabled, `onShutdown` is called and the interface is released.

Settings

Settings to be applied when an extension is enabled can be specified in the `settings` section. They are applied on the root of global settings. In case of any conflicts, the original settings are kept. It is recommended to use the path `exts/[extension_name]` for extension settings, but in general, any path can be used.

It is also good practice to document each setting in the `extension.toml` file, for greater discoverability of which settings a particular extension supports.

Tokens

When extension is enabled it sets tokens into the Carbonite `ITokens` interface with a path to the extension root folder. E.g. for the extension `omni.foo-bar`, the tokens `${omni.foo}` and `${omni.foo-bar}` are set.

Extensions Manager

Reloading

Extensions can be *hot reloaded*. The Extension system monitors the file system for changes to enabled extensions. If it finds any, the extensions are disabled and enabled again (which can involve reloading large parts of the dependency tree). This allows live editing of python code and recompilation of C++ plugins.

Use the `fswatcher.patterns` and `fswatcher.paths` config settings (see above) to control which files change triggers reloading.

Use the `reloadable` config setting to disable reloading. This will also block the reloading of all extensions this extension depends on. The extension can still be unloaded directly using the API.

New extensions can also be added and removed at runtime.

Extension interfaces

The Extension manager is implemented in `omni.ext.plugin`, with an interface: `omni::ext::IExtensions` (for C++), and `omni.ext` module (for python)

It is loaded by `omni.kit.app` and you can get an extension manager instance using its interface: `omni::kit::IApp` (for C++) and `omni.kit.app` (for python)

Runtime Information

At runtime, a user can query various pieces of information about each extension. Use `omni::ext::IExtensions::getExtensionDict()` to get a dictionary for each extension with all the relevant information. For python use `omni.ext.ExtensionManager.get_extension_dict()`.

This dictionary contains:

- Everything the extension.toml contains under the same path
- An additional `state` section which contains:
  - `state/enabled` (bool): Indicates if the extension is currently enabled.
  - `state/reloadable` (bool): Indicates if the extension can be reloaded (used in the UI to disable extension unloading/reloading)

Hooks

Both the C++ and python APIs for the Extension system provide a way to hook into certain actions/phases of the Extension System to enable extending it. If you register a hook like this:

```
def on_before_ext_enabled(self, ext_id: str, *_):
    pass

manager = omni.kit.app.get_app_interface().get_extension_manager()
self._hook = self._manager.get_hooks().create_extension_state_change_hook(
    self.on_before_ext_enabled,
    omni.ext.ExtensionStateChangeType.BEFORE_EXTENSION_ENABLE,
    ext_dict_path="python/pipapi",
    hook_name="python.pipapi",
)
```

...your callback will be called before each extension that contains the “python/pipapi” key in a config file is enabled. This allows us to write extensions that extend the Extension system. They can define their own configuration settings and react to them when extensions that contain those settings get loaded.

Extension API Code Examples

Enable Extension

```
# Extensions/Enable Extension
import omni.kit.app

manager = omni.kit.app.get_app().get_extension_manager()

# enable immediately
manager.set_extension_enabled_immediate("omni.kit.window.about", True)
print(manager.is_extension_enabled("omni.kit.window.about"))

# or next update (frame), multiple commands are be batched
manager.set_extension_enabled("omni.kit.window.about", True)
manager.set_extension_enabled("omni.kit.window.console", True)
```

Get All Extensions

```
# Extensions/Get All Extensions
import omni.kit.app

# there are a lot of extensions, print only first N entries in each loop
PRINT_ONLY_N = 10

# get all registered local extensions (enabled and disabled)
manager = omni.kit.app.get_app().get_extension_manager()
for ext in manager.get_extensions()[:PRINT_ONLY_N]:
    print(ext["id"], ext["package_id"], ext["name"], ext["version"], ext["path"]

# get all registered non-local extensions (from the registry)
# this call blocks to download registry (slow). You need to call it at least onc
manager.sync_registry()
for ext in manager.get_registry_extensions()[:PRINT_ONLY_N]:
    print(ext["id"], ext["package_id"], ext["name"], ext["version"], ext["path"]

# functions above print all versions of each extension. There is other API to ge
# "enabled_version" and "latest_version" contains the same dict as returned by f
for summary in manager.fetch_extension_summaries()[:PRINT_ONLY_N]:
    print(summary["fullname"], summary["flags"], summary["enabled_version"]["id"

# get all versions for particular extension
for ext in manager.fetch_extension_versions("omni.kit.window.script_editor"):
    print(ext["id"])
```

Get Extension Config

```
# Extensions/Get Extension Config
import omni.kit.app

manager = omni.kit.app.get_app().get_extension_manager()

# There could be multiple extensions with same name, but different version
# Extension id is: [ext name]-[ext version].
# Many functions accept extension id:
ext_id = manager.get_enabled_extension_id("omni.kit.window.script_editor")
data = manager.get_extension_dict(ext_id)

# Extension dict contains whole extension.toml as well as some runtime data:
# package section
print(data["package"])
# is enabled?
print(data["state/enabled"])
# resolved runtime dependencies
print(data["state/dependencies"])
# time it took to start it (ms)
print(data["state/startupTime"])

# can be converted to python dict for convenience and to prolong lifetime
data = data.get_dict()
print(type(data))
```

Get Extension Path

```
# Extensions/Get Extension Path
import omni.kit.app

manager = omni.kit.app.get_app().get_extension_manager()

# There could be multiple extensions with same name, but different version
# Extension id is: [ext name]-[ext version].
# Many functions accept extension id.
# You can get extension of enabled extension by name or by python module name:
ext_id = manager.get_enabled_extension_id("omni.kit.window.script_editor")
print(ext_id)
ext_id = manager.get_extension_id_by_module("omni.kit.window.script_editor")
print(ext_id)

# There are few ways to get fs path to extension:
print(manager.get_extension_path(ext_id))
print(manager.get_extension_dict(ext_id)["path"])
print(manager.get_extension_path_by_module("omni.kit.window.script_editor"))
```

Other Settings

`/app/extensions/disableStartup` (default: `false`)

Special mode where extensions are not started (the python and C++ startup functions are not called). Everything else will work as usual. One use-case might be to warm-up everything and get extensions downloaded. Another use case is getting python environment setup without starting anything.

`/app/extensions/precacheMode` (default: `false`)

Special mode where all dependencies are solved and extensions downloaded, then the app exits. It is useful for precaching all extensions before running an app to get everything downloaded and check that all dependencies are correct.

`/app/extensions/debugMode` (default: `false`)

Output more debug information into the info logging channel.

`/app/extensions/detailedSolverExplanation` (default: `false`)

Output more information after the solver finishes explaining why certain versions were chosen and what the available versions were (more costly).

`/app/extensions/registryEnabled` (default: `true`)

Disable falling back to the extension registry when the application couldn't resolve all its dependencies, and fail immediately.

`/app/extensions/skipPublishVerification` (default: `false`)

Skip extension verification before publishing. Use wisely.

`/app/extensions/excluded` (default: `[]`)

List of extensions to exclude from startup. Can be used with or without a version. Before solving the startup order, all of those extensions are removed from all dependencies.

`/app/extensions/preferLocalVersions` (default: `true`)

If `true`, prefer local extension versions over remote ones during dependency solving. Otherwise, all are treated equally, so it can become likely that newer versions are selected and downloaded.

`/app/extensions/syncRegistryOnStartup` (default: `false`)

Force sync with the registry on startup. Otherwise, the registry is only enabled if dependency solving fails (i.e. something is missing). The `--update-exts` command line switch enables this behavior.

`/app/extensions/publishExtraDict` (default: `{}`)

Extra data to write into the extension index root when published.

`/app/extensions/fsWatcherEnabled` (default: `true`)

Globally disable all filesystem watchers that the extension system creates.

`/app/extensions/mkdirExtFolders` (default: `true`)

Create non-existing extension folders when adding extension search path.

`/app/extensions/installUntrustedExtensions` (default: `false`)

Skip untrusted-extensions check when automatically installing dependencies and install anyway.

`/app/extensions/profileImportTime` (default: `false`)

Replace global `import` function with the one that sends events to `carb.profiler`. It makes all imported modules show up in a profiler. Similar to PYTHONPROFILEIMPORTTIME

`/app/extensions/fastImporter/enabled` (default: `true`)

Enable faster python importer, which doesn't rely on `sys.path` and manually scan extensions instead.

`/app/extensions/fastImporter/searchInTopNamespaceOnly` (default: `true`)

If `true` fast importer will skip search for python files in subfolders of the extension root that don't match module names defined in `[[python.module]]`. E.g. it won't usually look in any folder other than `[ext root]/omni`.

`/app/extensions/fastImporter/fileCache` (default: `false`)

Cache file existence checks in the fast importer. This speeds up startup time, but extensions python code can't be modified without cleaning the cache.

`/app/extensions/parallelPullEnabled` (default: `false`)

Enable parallel pulling of extensions from the registry.

Extension Registries

The Extension system supports adding external registry providers for publishing extensions to and pulling extensions from. By default, `Kit` comes with the omni.kit.registry.nucleus extension which adds support for Nucleus as an extension registry.

When an extension is enabled, the dependency solver resolves all dependencies. If a dependency is missing in the local cache, it will ask the registry for a particular extension and it will be downloaded/installed at runtime. Installation is just unpacking of a zip archive into the cache folder (`app/extensions/registryCache` setting).

The Extension system will only enable the Extension registry when it can't find all extensions locally. At that moment, it will try to enable any extensions specified in the setting: `app/extensions/registryExtension`, which by default is `omni.kit.registry.nucleus`.

The Registry system can be completely disabled with the `app/extensions/registryEnabled` setting.

The Extension manager provides an API to add other extension registries and query any existing ones (`omni::ext::IExtensions::addRegistryProvider`, `omni::ext::IExtensions::getRegistryProviderCount` etc).

Multiple registries can be configured to be used at the same time. They are uniquely identified with a name. Setting `app/extensions/registryPublishDefault` sets which one to use by default when publishing and unpublishing extensions. The API provides a way to explicitly pass the registry to use.

Publishing Extensions

To properly publish your extension to the registry use the publishing tool, refer to: Publishing Extensions Guide

Alternatively, `kit.exe --publish` CLI command can be used during development:

Example: `> kit.exe --publish omni.my.ext-tag`

If there is more than one version of this extension available, it will produce an error saying that you need to specify which one to publish.

Example: `> kit.exe --publish omni.my.ext-tag-1.2.0`

To specify the registry to publish to, override the default registry name:

Example: `> kit.exe --publish omni.my.ext-tag-1.2.0 --/app/extensions/registryPublishDefault="kit/mr"`

If the extension already exists in the registry, it will fail. To force overwriting, use the additional `--publish-overwrite` argument:

Example: `> kit.exe --publish omni.my.ext --publish-overwrite`

The version must be specified in a config file for publishing to succeed.

All `[package.target]` config subfields are filled in automatically if unspecified:

- If the extension config has the `[native]` field `[package.target.platform]` or `[package.target.config]`, they are filled with the current platform information.
- If the extension config has the `[native]` and `[python]` fields, the field `[package.target.python]` is filled with the current python version.
- If the `/app/extensions/writeTarget/kitHash` setting is true, the field `[package.target.kitHash]` is filled with the current kit githash.

An Extension package name will look like this: `[ext_name]-[ext_tag]-[major].[minor].[patch]-[prerelease]+[build]`. Where:

- `[ext_name]-[ext_tag]` is the extension name (initially coming from the extension folder).
- `[ext_tag]-[major].[minor].[patch]-[prerelease]` if `[package.version]` field of a config specifies.
- `[build]` is composed from the `[package.target]]` field.

Unpublishing Extensions

For any modification, it is recommended to bump a version and release a new extension. All previously published versions are considered immutable. Users can have them already cached on their system, older builds depend on them etc. However, in rare cases, it might be necessary to prevent users from getting a particular version of an extension. For example, if it is found to be malicious or has a critical bug. In that case, it is possible to unpublish an extension.

Unpublishing doesn't fully remove an extension from the registry, it just marks an extension as "yanked". This way those apps who request exactly that version will still get it, but it won't be picked by the dependency solver if not specified exactly. Older builds won't break, but it would prevent new users from getting that version. It is similar to [cargo yank] [https://doc.rust-lang.org/cargo/commands/cargo-yank.html] or npm deprecate.

To unpublish an extension, use the `--unpublish` CLI argument:

Example: `> kit.exe --unpublish omni.my.ext-tag-1.2.0`

This unpublishes one package. If an extension has multiple packages (e.g. for different platforms) that command will fail and ask to specify which package to unpublish. You need to unpublish each one separately by specifying the full package id:

Example: `> kit.exe --unpublish omni.foo-1.2.3+wx64.r.cp310`

To fully remove an extension from the registry (not recommended) you can pass `--/exts/omni.kit.registry.nucleus/yankWhenUnpublishing=0`.

Pulling Extensions

There are multiple ways to get Extensions (both new Extensions and updated versions of existing Extensions) from a registry:

1. Use the UI provided by this extension: `omni.kit.window.extensions`

2. If any extensions specified in the app config file - or required through dependencies - are missing from the local cache, the system will attempt to sync with the registry and pull them. That means, if you have version “1.2.0” of an Extension locally, it won’t be updated to “1.2.1” automatically, because “1.2.0” satisfies the dependencies. To force an update run Kit with `--update-exts` flag:

Example: `> kit.exe --update-exts`

Pre-downloading Extensions

You can also run Kit without starting any extensions. The benefit of doing this is that they will be downloaded and cached for the next run. To do that run Kit with `--ext-precache-mode` flag:

Last updated on Jun 12, 2026.